

Teste Técnico — Java + Spring Boot

Aplicação REST para Controle de Dívidas

1. Objetivo

Desenvolver uma aplicação backend utilizando Java e Spring Boot para realizar o cadastro e o gerenciamento de dívidas de devedores por meio de uma API REST.

2. Requisitos funcionais

- **RF01 — Cadastrar dívida:** permitir o cadastro de uma nova dívida.
- **RF02 — Consultar dívida:** permitir a consulta de uma dívida por seu identificador.
- **RF03 — Listar dívidas:** permitir a consulta de todas as dívidas cadastradas.
- **RF04 — Atualizar dívida:** permitir a alteração dos dados de uma dívida existente.
- **RF05 — Excluir dívida:** permitir a exclusão de uma dívida existente.
- **RF06 — CPF do devedor:** armazenar o CPF associado à dívida.

3. Dados da dívida

Cada dívida deve possuir, no mínimo, os seguintes dados:

Campo	Tipo sugerido	Descrição
id	Long	Identificador único da dívida.
cpfDevedor	String	CPF do devedor.
valorPego	BigDecimal	Valor originalmente tomado pelo devedor.
valorComJuros	BigDecimal	Valor da dívida considerando os juros.
valorComDesconto	BigDecimal	Valor da dívida considerando o desconto.

4. API REST

A aplicação deverá disponibilizar endpoints REST para as operações de CRUD. A estrutura abaixo é uma sugestão; o candidato pode organizar os endpoints de forma equivalente, desde que as operações sejam atendidas.

Método	Endpoint	Finalidade
POST	/dividas	Cadastrar uma dívida.
GET	/dividas/{id}	Consultar uma dívida pelo ID.
GET	/dividas	Listar todas as dívidas.
PUT	/dividas/{id}	Atualizar uma dívida.

Método	Endpoint	Finalidade
DELETE	/dividas/{id}	Excluir uma dívida.

5. Exemplo de requisição

Exemplo de corpo para criação de uma dívida:

```
{  
  "cpfDevedor": "12345678900",  
  "valorPego": 1000.00,  
  "valorComJuros": 1200.00,  
  "valorComDesconto": 1100.00  
}
```

6. Requisitos técnicos

- Utilizar **Java**.
- Utilizar **Spring Boot**.
- Disponibilizar uma API REST.
- Utilizar persistência de dados. Pode ser utilizado banco de dados relacional de sua preferência.
- Utilizar **BigDecimal** para os valores monetários.
- Organizar o projeto de forma clara, separando responsabilidades entre camadas, quando aplicável.
- Utilizar tratamento adequado dos códigos HTTP e dos erros da API.
- Disponibilizar instruções para execução do projeto.

7. Validações esperadas

- CPF deve ser informado.
- Os valores monetários devem ser válidos e não negativos.
- Uma dívida inexistente deve resultar em resposta HTTP apropriada.
- Operações de atualização e exclusão devem validar a existência do registro.
- A API deve retornar respostas HTTP coerentes com cada operação.

8. Diferenciais

Os itens abaixo não são obrigatórios, mas podem ser utilizados para demonstrar conhecimentos adicionais:

- Testes unitários e/ou testes de integração.
- Documentação da API com OpenAPI/Swagger.
- Docker/Docker Compose.
- Uso de migrations para o banco de dados.
- Paginação na listagem de dívidas.
- Filtros ou consultas por CPF.
- Validações utilizando Bean Validation.
- Tratamento global de exceções.
- Logs e organização adequada das configurações da aplicação.

9. Entrega

Entregar o código-fonte do projeto em um repositório Git, contendo um README com as instruções necessárias para configurar, executar e testar a aplicação.

O README deve informar também quais tecnologias foram utilizadas e, caso necessário, os parâmetros de configuração do banco de dados.

10. Critérios de avaliação

Critério	O que será observado
Funcionamento	CRUD completo e API funcionando corretamente.
Código	Clareza, organização, legibilidade e boas práticas.
Spring Boot	Uso adequado dos recursos do framework.
Persistência	Modelagem e acesso aos dados.
API REST	Endpoints, métodos HTTP, status codes e contratos.
Validações	Tratamento de entradas inválidas e cenários de erro.
Testes	Cobertura e qualidade dos testes, quando apresentados.
Documentação	Facilidade para executar e compreender o projeto.

Observação: O candidato pode tomar decisões de implementação diferentes das sugestões deste documento, desde que os requisitos principais sejam atendidos.